The a_value_within matcher is an alias for be_within that acts identically, except for how it describes itself. RSpec provides a number of aliases like this for each of the built-in matchers.[1] As we'll see in *Defining Matcher Aliases*, on page 218, it's trivial to define your own aliases as well.

Embedding Matchers in Array and Hash Data Structures

In addition to passing one matcher into another, you can also embed matchers within an array at any position, or within a hash in place of a value. RSpec will compare the corresponding items using ===. This technique works at any level of nesting. Let's look at an example:

10-exploring-rspec-expectations/04/composing_matchers.rb

```
presidents = [
  { name: 'George Washington', birth_year: 1732 },
  { name: 'John Adams',        birth_year: 1735 },
  { name: 'Thomas Jefferson',  birth_year: 1743 },
  # ...
]
expect(presidents).to start_with(
  { name: 'George Washington', birth_year: a_value_between(1730, 1740) },
  { name: 'John Adams',        birth_year: a_value_between(1730, 1740) }
)
```

Here, we're using a_value_between(1730, 1740) for the birth years of George Washington and John Adams instead of a specific number. Of course, we know their birth years were exactly 1732 and 1735, respectively. But not all your real-world values are going to be so precise. If you test for behavior more specific than what you actually need, or if you're testing nondeterministic logic, your specs may break if the implementation changes slightly.

This ability to compose matchers—by passing them into one another, or by embedding them in data structures—lets you be as precise or as vague as you need to be. When you specify exactly the behavior you expect (and nothing more), your tests become less brittle.

Combining Matchers With Logical and/or Operators

There's another way to combine matchers: *compound matcher expressions*. Every built-in matcher has two methods (and and or) that allow you to logically combine any two matchers into a compound matcher:

10-exploring-rspec-expectations/04/composing_matchers.rb

```
alphabet = ('a'..'z').to_a
expect(alphabet).to start_with('a').and end_with('z')
```

1. http://rspec.info/documentation/3.6/rspec-expectations/RSpec/Matchers.html